

Agentic AI for Product Development

Activity 05: Design an Agent Workflow Canvas

TGS-2024045799 • v7.0

Map a product-discovery task into an agentic loop with tools, state, stop conditions and human gates.

1. Purpose and deliverable

Map a product-discovery task into an agentic loop with tools, state, stop conditions and human gates.

Required output: Workflow canvas and responsibility matrix

Acceptance: Every stage has an owner and observable output; consequential actions stop for human review.

2. Folder inventory

brief.yaml — machine-readable scope and decision contract

prompt.md — bounded agent instruction

data/mock-data.csv and mock-context.json — synthetic evidence

solution/expected-output.json — trainer reference shape

scripts/verify_activity.py — deterministic completeness check

evidence/checklist.md — learner sign-off record

3. Detailed procedure

1. Read brief.yaml and restate the product decision, output and acceptance rule.
2. Inspect the CSV and JSON inputs; list source IDs, dates, contradictions and missing evidence.
3. Open prompt.md and select an approved agentic AI builder. Do not add live data or integrations.
4. Ask the agent to plan the workflow before producing the artifact. Check roles, tools, data and stop conditions.
5. Execute the activity-specific workflow: Define goal → Map stages → Assign tools → Set schemas → Add gates → Set stop.
6. Require citations to source IDs and label every inference or assumption.
7. Review the output at the human gate; compare it with the acceptance rule and reject unsupported claims.
8. Save the artifact and evidence inside a learner working copy of this folder.
9. Run `python3 scripts/verify_activity.py` and retain the JSON result.
10. Complete evidence/checklist.md and record PASS, REPAIR or STOP with a named owner.

4. Troubleshooting

Missing citations: require row-level source IDs and rerun only the unsupported section.

Unstable recommendation: freeze criteria, rerun the same input and perform sensitivity analysis.

Verifier failure: compare the missing file or field with brief.yaml, repair it and rerun.

Unsafe action proposed: stop, remove live/external access and return the decision to the human owner.

5. Verification

Pass condition: Every stage has an owner and observable output; consequential actions stop for human review.

The deterministic verifier checks package completeness; the human reviewer checks evidence quality and decision fitness.